Jarosław Kuchta

Jakość Systemów Informatycznych

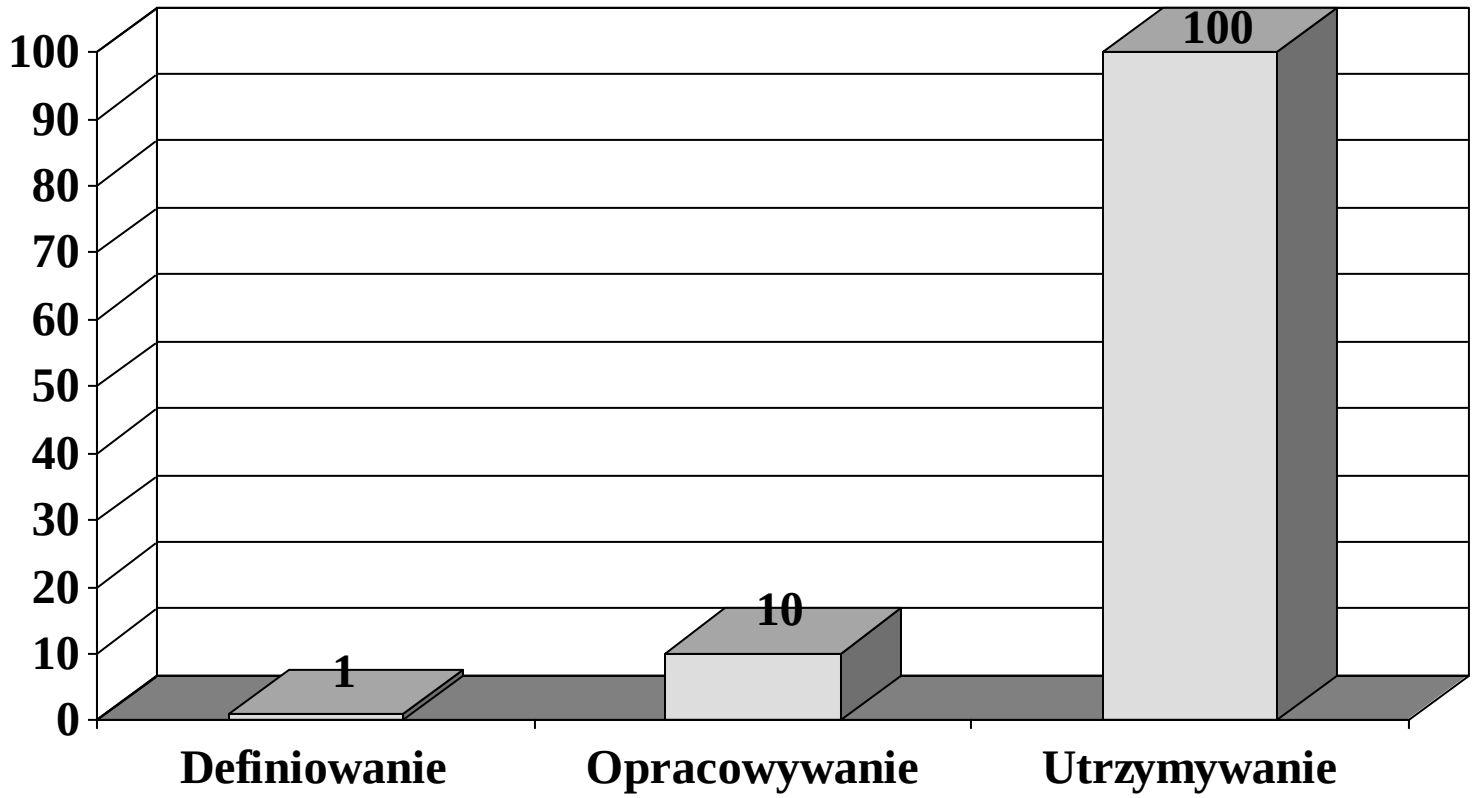
Jakość Oprogramowania

Jakość w procesie wytwarzania oprogramowania

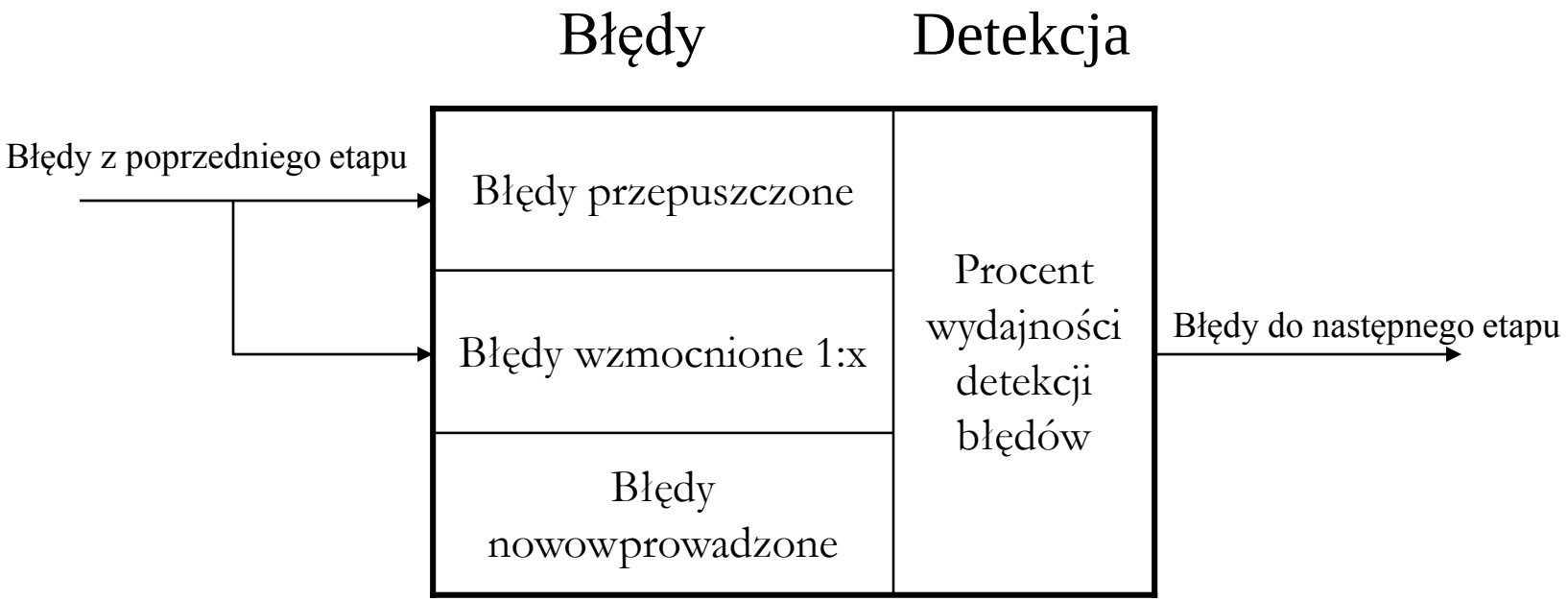
http://www.eti.pg.gda.pl/katedry/kask/pracownicy/Jaroslaw.Kuchta/Jakosc/

J.Kuchta@eti.pg.gda.pl

Względny koszt wprowadzania zmian w zależności od fazy realizacji projektu



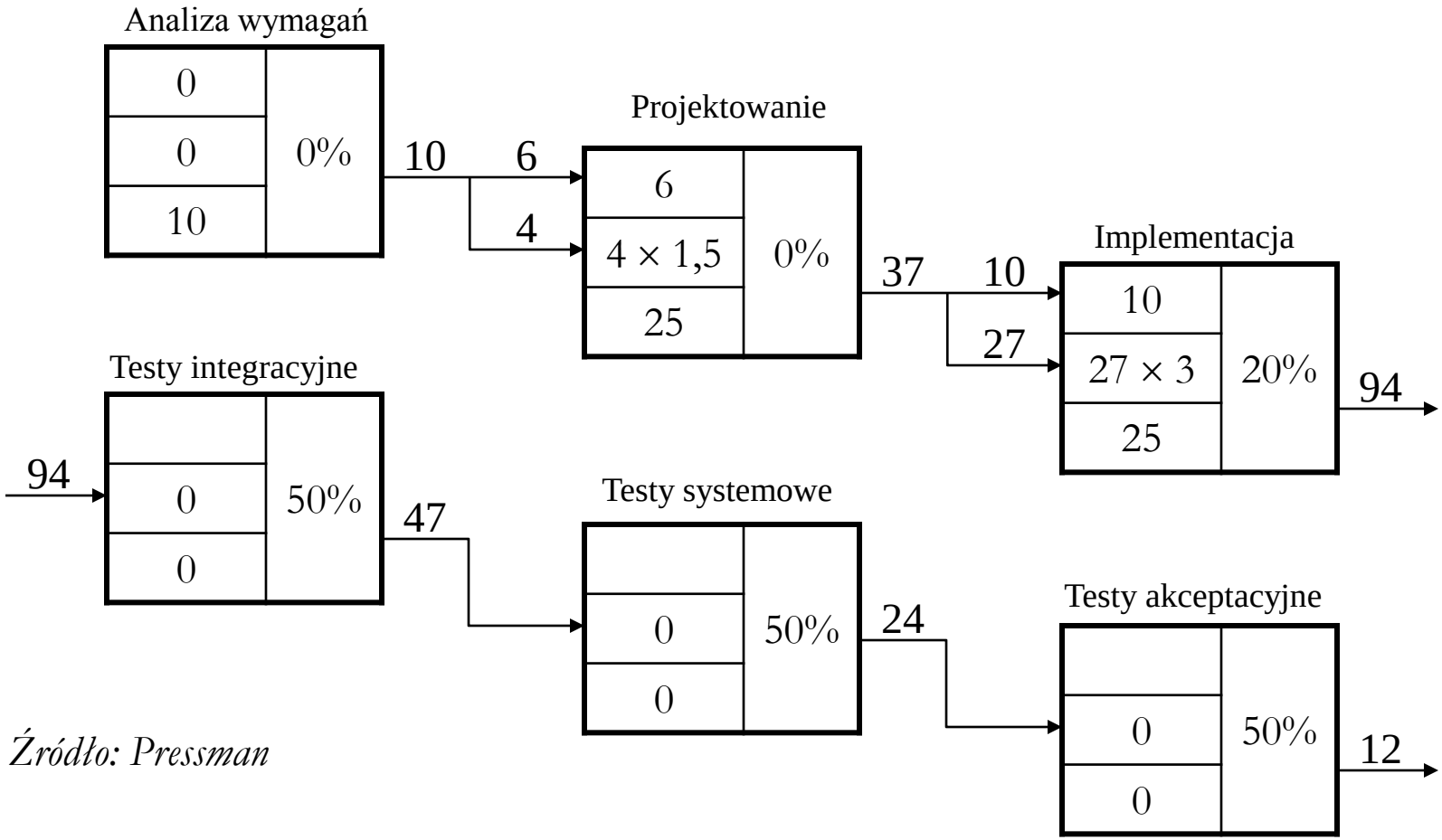
Model wzmocnienia błędów



Źródło: Pressman

Efekt wzmocnienia błędów

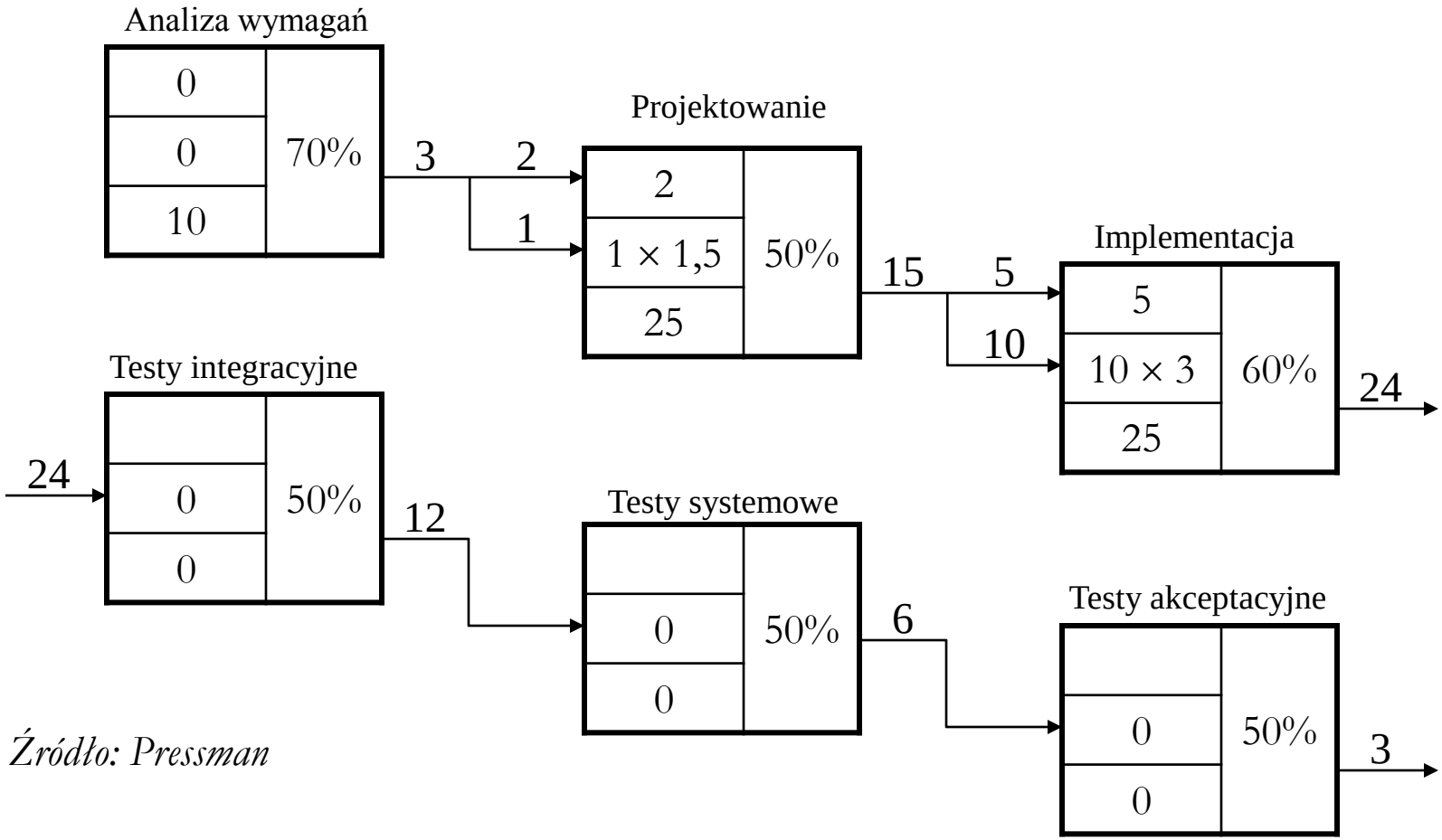
– kontrola jakości przy testowaniu



Źródło: Pressman

Efekt wzmocnienia błędów

– kontrola jakości w całym procesie



Źródło: Pressman

Porównanie kosztów wytwarzania

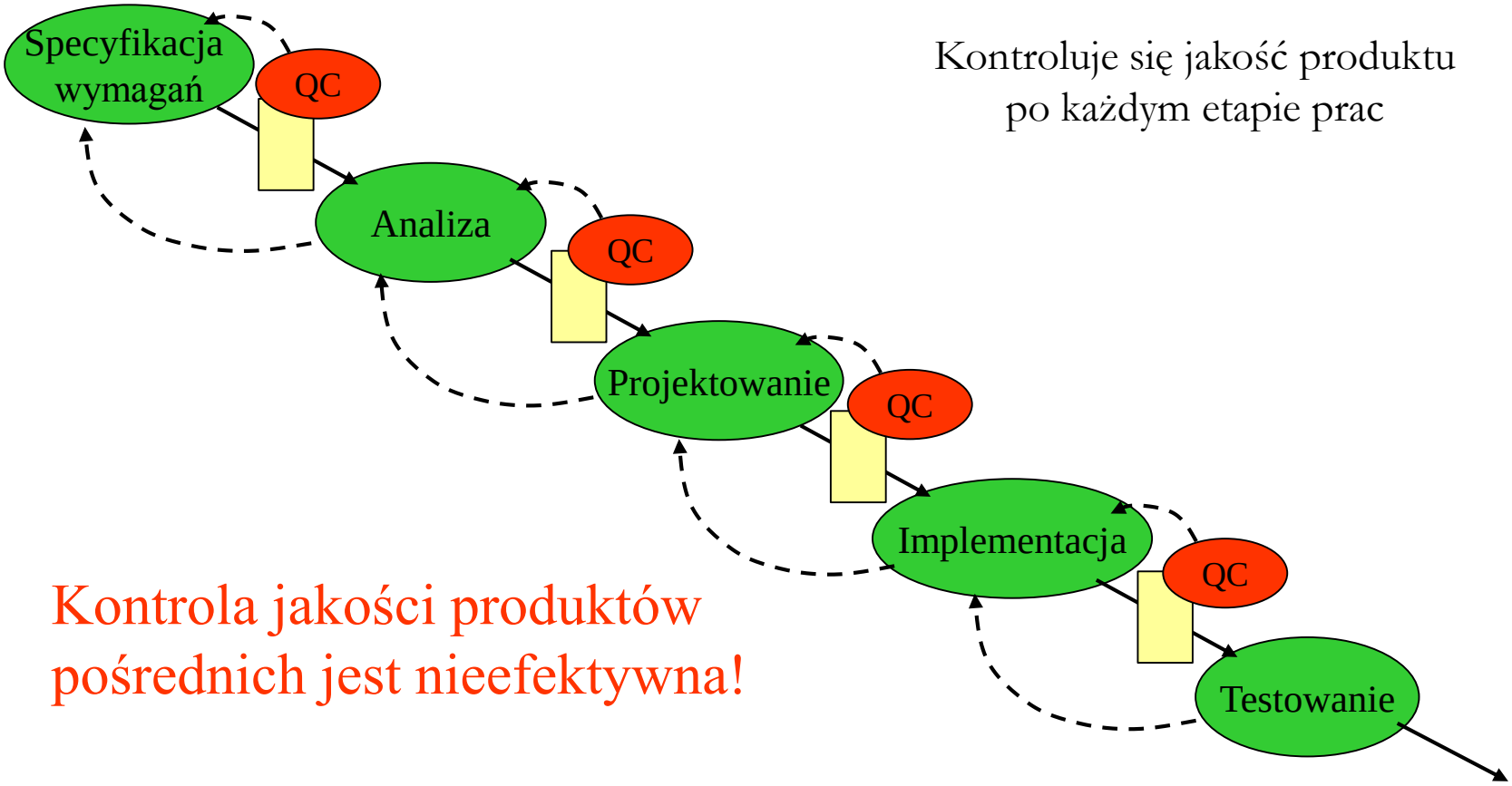
Błędy wykryte	Liczba błędów	Koszt jednostkowy	Koszt sumaryczny
Kontrola jakości przy testowaniu			
Przed testowaniem	22	6,5	143
Podczas testowania	82	15	1230
Po wydaniu	12	67	804
Razem			**2177**
Kontrola jakości w całym procesie			
Podczas wytwarzania	22	1,5	33
Przed testowaniem	36	6,5	234
Podczas testowania	15	15	225
Po wydaniu	3	67	202
Razem			**693**

Źródło: Pressman

Ewolucja polityki jakościowej

- **Kontrola jakości (QC – *Quality Control*)**
  – lata 1970-te – kontrolowanie jedynie jakości produktu końcowego
- **Sterowanie jakością (QC – *Quality Control*)**
  – lata 1980-te – kontrolowanie jakości produktów pośrednich
- **Zapewnienie jakości (SQA – *Software Quality Assurance*)**
  – lata 1990-te – opracowanie procedur zapewniających jakość na każdym etapie
- **Zarządzanie jakością (TQM – *Total Quality Management*)** – lata 2000 – przeniesienie ciężaru zapewnienia jakości z inżynierów na całą organizację (zarządzanie)

Proces dla Quality Control



Kontroluje się jakość produktu po każdym etapie prac

Kontrola jakości produktów pośrednich jest nieefektywna!

Software Quality Assurance

- *jest to planowy i usystematyzowany zbiór akcji wymaganych dla zapewnienia jakości w oprogramowaniu*
- powołuje się **grupę SQA** (inżynierowie, kierownicy, klienci, sprzedawcy i inni), która przygląda się powstającemu oprogramowaniu z punktu widzenia klienta:
  - Czy oprogramowanie odpowiada czynnikom jakości?
  - Czy oprogramowanie powstaje zgodnie z wcześniej ustanowionymi standardami?
  - Czy techniczne dyscypliny odpowiednio wypełniają swoje role w aspekcie aktywności SQA?

Aktywności SQA

- Stosowanie metod technicznych
- Przeprowadzanie formalnych przeglądów technicznych (***FTR – Formal Technical Review***)
- Testowanie oprogramowania
- Wymuszenie standardów
- Kontrolowanie zmian
- Wykonywanie pomiarów
- Zapisywanie i raportowanie

Cele FTR

- wykrycie błędów w funkcjach, logice lub implementacji oprogramowania w dowolnej jego reprezentacji,
- sprawdzenie, czy przeglądane oprogramowanie jest zgodne z wymaganiami,
- upewnienie się, że reprezentacja oprogramowania jest zgodna z wcześniej zdefiniowanymi standardami,
- uzyskanie oprogramowania opracowanego w jednolity sposób,
- sprawienie, by projekty były łatwiejsze w utrzymaniu.

Rodzaje FTR

- przegląd (*walkthrough*)
  - przejrzenie treści dokumentu zgodnie z jego logicznym uporządkowaniem (np. odczytanie dokumentu)
- inspekcje
  - przejrzenie dokumentu zgodnie z listą kontrolną

Lista kontrolna dla specyfikacji wymagań

- Czy główne funkcje są zdefiniowane w sposób powiązany i jednoznaczny?
- Czy interfejsy między elementami systemowymi są zdefiniowane?
- Czy ustalono granice wydajności dla całego systemu oraz dla każdego jego elementu?
- Czy ustalono ograniczenia projektowe dla każdego elementu?
- Czy wybrane zostało najlepsze rozwiązanie?
- Czy rozwiązanie jest technologicznie możliwe?
- Czy opracowano mechanizm walidacji i weryfikacji systemu?
- Czy zachowano spójność pomiędzy wszystkimi elementami systemu?

Lista kontrolna dla analizy

- Czy analiza dziedziny problemu jest kompletna, zwarta i dokładna?
- Czy zakończono partycjonowanie problemu?
- Czy model danych odpowiada obiektom danych, ich atrybutom i relacjom?
- Czy wszystkie wymagania znalazły odwzorowanie w modelu?
- Czy wykonano prototyp dla użytkownika?
- Czy wydajność jest osiągalna w aspekcie ograniczeń wprowadzanych przez inne elementy systemu?
- Czy kryteria walidacyjne są kompletne?

Lista kontrolna dla projektowania (1)

- Czy wymagania softwerowe znalazły odwzorowanie w architekturze oprogramowania?
- Czy zastosowano odpowiednią modularyzację? Czy moduły są funkcjonalnie niezależne?
- Czy zdefiniowano interfejsy dla modułów i elementów systemów zewnętrznych?
- Czy struktura danych jest spójna z dziedziną informacji?
- Czy struktura danych jest spójna ze specyfikacją wymagań?
- Czy uwzględniono łatwość pielęgnacji?
- Czy jawnie uwzględniono czynniki jakości?

Lista kontrolna dla projektowania (2)

- Czy algorytmy odpowiadają wymaganym funkcjom?
- Czy algorytmy są logicznie poprawne?
- Czy interfejs jest spójny z projektem architektury?
- Czy logiczna złożoność jest rozsądna?
- Czy określono obsługę błędów i ochronę przed błędami?
- Czy lokalne struktury danych są poprawnie zdefiniowane?
- Czy zachowano konstrukcje programowania strukturalnego?
- Czy szczegółowość projektowania jest odpowiednia dla języka implementacji?
- Czy uwzględniono łatwość pielęgnacji?

Lista kontrolna dla implementacji

- Czy projekt został odpowiednio przetłumaczony na kod?
- Czy zachowano zgodność ze standardami kodowania w zakresie stylu języka, komentarzy, prologów modułów?
- Czy nie ma niepoprawnych lub niejednoznacznych komentarzy?
- Czy typy danych zostały odpowiednio dobrane?
- Czy stałe fizyczne zostały poprawnie zdefiniowane?

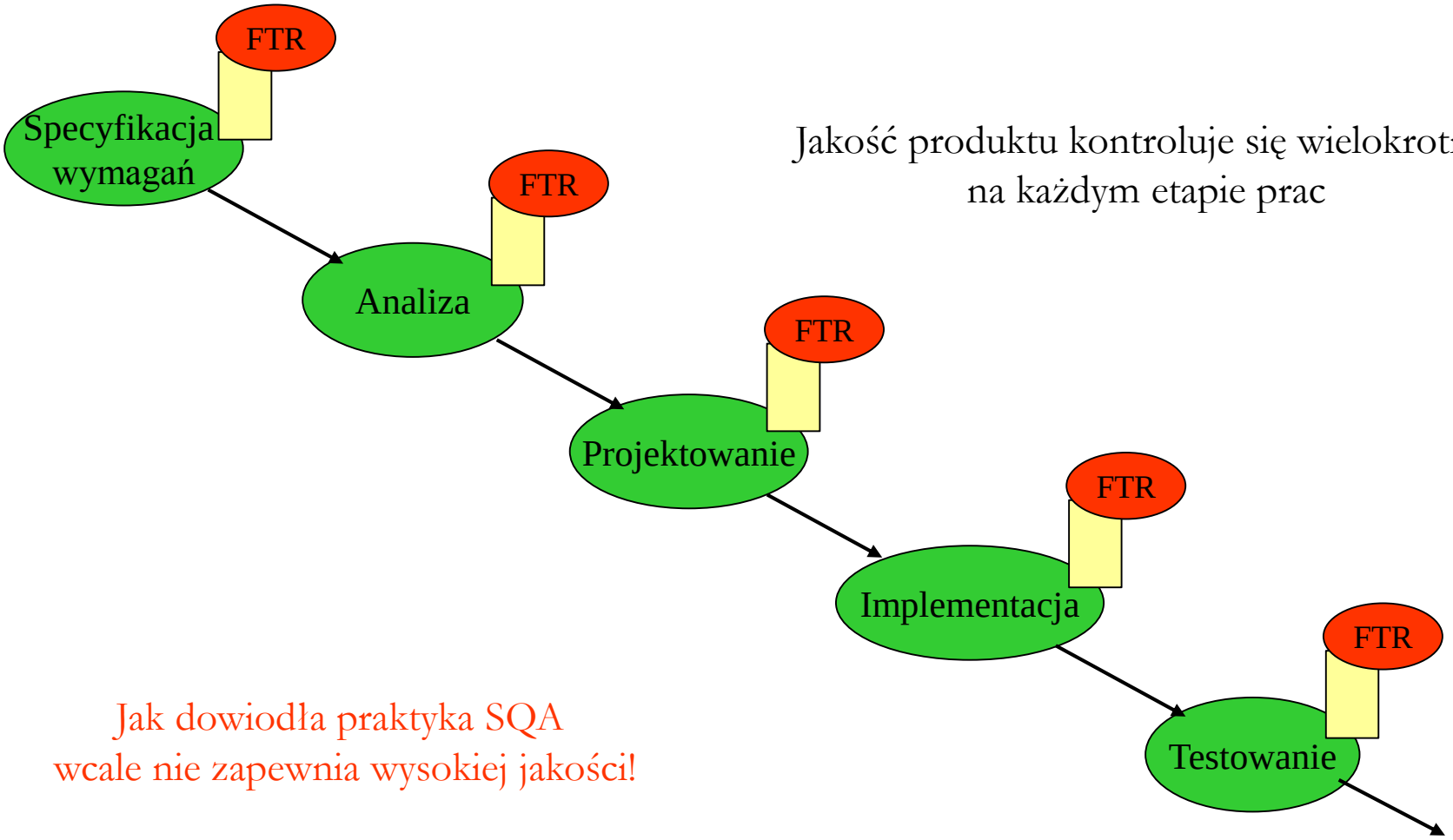
Lista kontrolna dla testowania (1)

- Czy główne fazy testowania zostały poprawnie zidentyfikowane i przeprowadzone?
- Czy zachowano zgodność kryteriów walidacji ze specyfikacją wymagań?
- Czy główne funkcje zademonstrowano wcześnie?
- Czy zasoby testów zostały zidentyfikowane i są dostępne?
- Czy ustalono sposób zapisywania wyników testów?
- Czy moduły sterujące testami i moduły zastępcze testów zostały zidentyfikowane?
- Czy określono testy obciążeniowe oprogramowania?

Lista kontrolna dla testowania (2)

- Czy przeprowadzono testowanie metodą białej i czarnej skrzynki?
- Czy wszystkie niezależne ścieżki logiczne zostały przetestowane?
- Czy przypadki testowe zostały zidentyfikowane i wylistowane z oczekiwanymi rezultatami?
- Czy obsługa błędów została przetestowana?
- Czy wartości graniczne zostały przetestowane?
- Czy przetestowano timing i wydajność?
- Czy określono dopuszczalne odchylenia od oczekiwanych rezultatów?

Proces dla SQA



Jakość produktu kontroluje się wielokrotnie na każdym etapie prac

Jak dowiodła praktyka SQA wcale nie zapewnia wysokiej jakości!

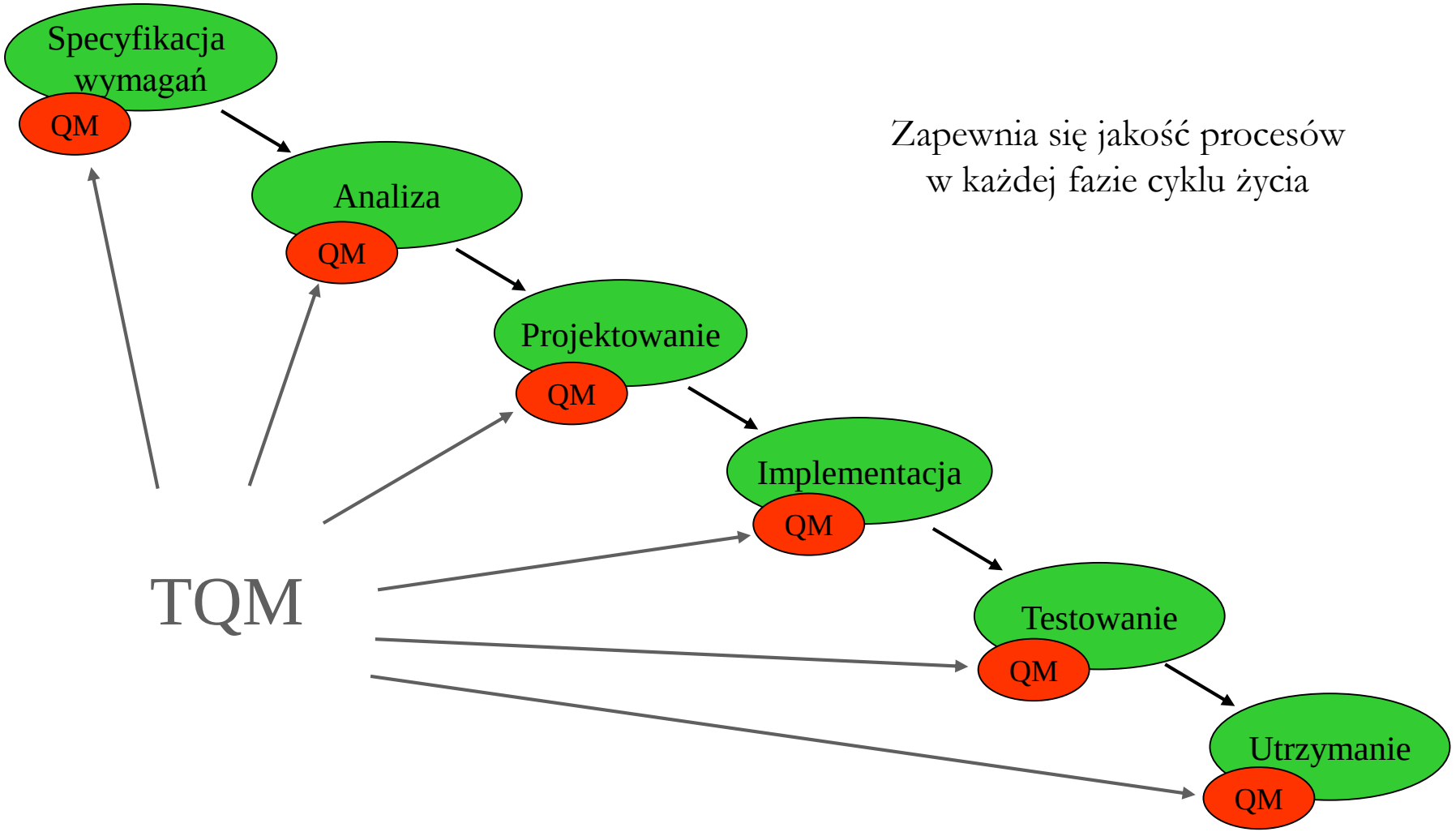
Co to jest TQM?

- *Total Quality Management* – jest to zbiór działań sprawiających, że każdy członek organizacji (od dyrektora do sprzątaczki) rozumie, jakie są oczekiwania klientów tej organizacji i dąży do spełnienia tych oczekiwań
- Rozumienie i spełnianie oczekiwań klientów jest wyzwaniem dla organizacji i wymaga odpowiednich procesów.

Naczelne zasady TQM

- Jakość może i musi być zarządzana.
- Każdy ma swojego klienta i swojego dostawcę.
- Procesy, a nie ludzie stanowią problem.
- Każdy pracownik jest odpowiedzialny za jakość.
- Problemom trzeba zapobiegać, a nie tylko rozwiązywać
- Jakość musi być mierzona.
- Poprawa jakości musi być stała
- Standard jakości jest wolny od defektów.
- Cele są oparte o wymagania, a nie negocjowane.
- Koszty są ukryte w całym cyklu życia, a nie tylko w wytwarzaniu.
- Kierownictwo musi być zaangażowane i musi przewodzić.
- Działania na rzecz poprawy jakości muszą być planowane i organizowane.

Podejście procesowe TQM



Zapewnia się jakość procesów w każdej fazie cyklu życia

TQM w inżynierii oprogramowania

- W fazie specyfikacji wymagań konieczne jest określenie potrzeb wszystkich użytkowników, w tym użytkowników informacji generowanej przez oprogramowanie, użytkowników wprowadzających dane i dostawców wewnętrznych.
- W fazie analizy i projektowania konieczne jest przeglądanie projektów przez użytkowników.
- W fazie implementacji konieczne jest przeprowadzanie przeglądów wzajemnych (*peer review*).
- W fazie testowania konieczne jest przeprowadzenie testowania przez klientów i ocena dokumentacji.
- W fazie utrzymania przeprowadza się szkolenia i stale doskonali procesy projektowe i relacje z klientami.

Literatura

- Pressman R.S., *Software engineering. A practitioner's approach*, McGraw-Hill, International Edition, 1992
- Górski J. et al., *Inżynieria oprogramowania w projekcie informatycznym*, wyd. Mikom, Warszawa, 2000
- Dahlgaard J. J., Kristensen K., Kanji G. K. - *Fundamentals of Total Quality Management*, Polish edition by PWN, 2000.
- Grudowski P., Kolman R., Meller A., Preihs J. - *Zarządzanie jakością (Quality Management)*, Wydawnictwo Politechniki Gdanskiej, 1996.